AUTOSAR HLD Document Analysis Assistant

A tool for reviewing AUTOSAR High-Level Design documents

Capstone Project - Technical Report

Field	Details
Student name	Kurapati Jaya Devesh
Register number	127003137
college	SASTRA
Repository	https://github.com/JayaDeveshKurapati/autosar-hld-assistant
Date	15 October 2026

Contents

1. Problem Statement and Intended Users 3
1.1 Problem 3
1.2 Objective 3
1.3 Intended users 3
2. Input Data and Knowledge Base 3
3. Solution Architecture 4
3.1 Design choice 4
3.2 Components 4
3.3 Pre-trained model usage 5
4. Implementation and Configuration 5
4.1 Q&A workflow 5
4.2 Review and approval workflow 6
4.3 Consistency rules 6
4.4 Setup and configuration 6
5. Evaluation Steps and Results 6
5.1 Method 6
5.2 Results - analysis of the sample documents 7
5.3 Results - Q&A probe (v1, extractive mode) 7
5.4 Results - revision comparison (v1 to v2) 7
5.5 Built-in metrics 8
5.6 Limitations observed 8
6. Responsible AI Measures 8
7. Innovation Highlights 8
8. Conclusion and Future Work 9
References and Third-Party Components 9
Academic Integrity and AI-Tool Usage Declaration ..... **Error! Bookmark not defined.**
A. Academic Integrity Declaration ..... **Error! Bookmark not defined.**
B. AI-Tool Usage Declaration ..... **Error! Bookmark not defined.**
Appendix . Submission Checklist 10

1. Problem Statement and Intended Users

1.1 Problem

AUTOSAR High-Level Design (HLD) documents describe software components, ports, interfaces, signals, runnables and timing for automotive ECUs. They are long, change often between revisions, and are reviewed by hand. Typical defects - a data-type disagreement between two connected ports, an interface that is referenced but never defined, a runnable period that contradicts a latency requirement - are easy to miss and expensive when found late in integration. Engineers also lose time searching these documents to answer simple questions such as "who provides WheelSpeed?" or "what changed between v1 and v2?".

1.2 Objective

Build an assistant that ingests HLD documents (or ARXML), extracts the architecture into a structured model, answers questions with mandatory citations, flags inconsistencies as candidates for human review, and exports a review report - while never presenting an unverified machine output as a design decision.

1.3 Intended users

- **Software architects** - check a new HLD revision for consistency before release.
- **Design reviewers / safety engineers** - review findings, record decisions, sign off.
- **Integration and test engineers** - obtain traced test-case skeletons and see change impact.
- **CI maintainers** - use the CLI as a gate that fails the build on design errors.

2. Input Data and Knowledge Base

The knowledge base is the set of documents the user uploads into a project; nothing is pre-trained on external data. Supported inputs:

Format	Handling	Notes
PDF	Text per page (pypdf)	Page numbers preserved for citations; no OCR
DOCX	python-docx; headings auto-numbered, tables flattened	Section structure retained
TXT / MD	Numbered-heading parser	Convention in sample_data/
ARXML	XML parser: interfaces, SWCs, ports, runnables, timing events, composition connectors	Most robust path for real projects

Sample knowledge base shipped with the project (a Brake Control ECU):

- **brake_system_hld_v1.txt** - baseline revision, deliberately seeded with defects (type mismatches, undefined interface and component).
- **brake_system_hld_v2.txt** - revised design with a new VehicleSpeedEstimator component and changed timing.
- **brake_system.arxml** - the same system in ARXML form.

Each document is split into section-level chunks. Every chunk and every extracted entity keeps its document, version, page, section and original evidence text, which is what makes citation possible.

3. Solution Architecture

3.1 Design choice

The solution is a hybrid: a deterministic extraction-and-rules layer for design checking, plus a retrieval-augmented generation (RAG) layer for question answering. Checks are rule-based on purpose - a reviewer must be able to trace why a finding exists. The RAG layer is retrieval-first: it works without any LLM (extractive answers) and can optionally call an OpenAI-compatible model (for example Ollama or vLLM) whose prompt is restricted to the retrieved excerpts.

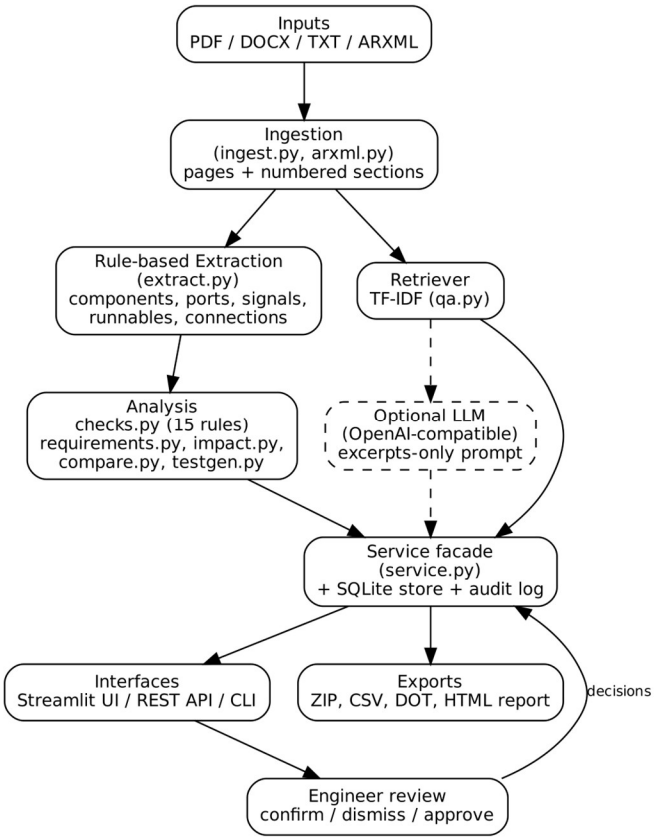


Figure 1. System architecture

3.2 Components

Module	Responsibility
ingest.py / arxml.py	Read PDF, DOCX, TXT, ARXML into paged, sectioned text and structured ARXML entities
extract.py	Rule-based extraction of components, interfaces, ports, signals, runnables, dependencies, connections - each with page, section, evidence
checks.py	15 consistency rules producing findings (error / warning / info)
requirements.py	Finds "shall/must" statements, types them (timing, safety, functional), parses numeric limits, links them to entities
impact.py	Up/downstream change impact, dependency cycles, source-to-sink chains with a coarse latency estimate

Module	Responsibility
compare.py	Structural diff and section text diff with value-change detection (20 ms -> 15 ms)
testgen.py	Traced test-case skeletons
qa.py	TF-IDF retriever, confidence scoring, optional LLM, extractive fallback, refusal
reports.py / reportgen.py	CSV/JSON/DOT ZIP export and printable HTML review report
store.py / service.py	SQLite persistence, audit log, single service facade used by UI, API and CLI
app.py / api.py / cli.py	Streamlit UI, FastAPI REST service, command-line / CI gate

3.3 Pre-trained model usage

No model is trained or fine-tuned. Retrieval uses scikit-learn TF-IDF with a custom identifier-aware tokenizer (it splits CamelCase and snake_case such as WheelSpeedProvider into wheel/speed/provider while keeping the whole identifier searchable) with unigrams and bigrams. Generation is optional and delegated to whichever pre-trained LLM the user configures through HLD_LLM_URL and HLD_LLM_MODEL; the shipped evaluation below was run in extractive mode with no LLM.

4. Implementation and Configuration

4.1 Q&A workflow

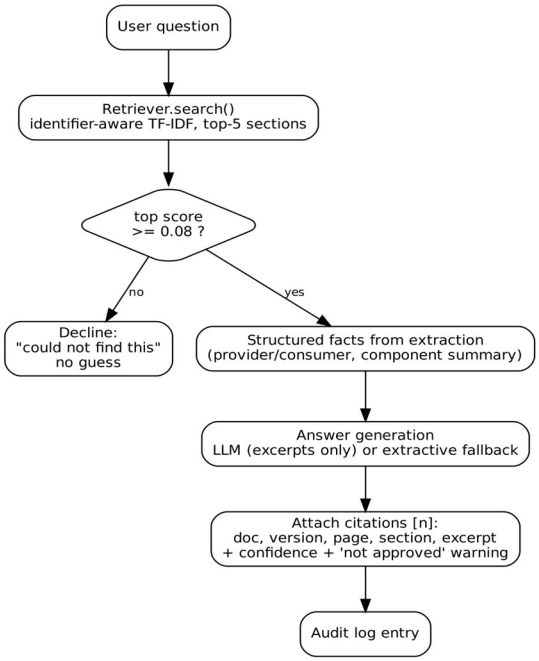


Figure 2. Question answering workflow with refusal path

- Retrieval returns the top 5 section chunks. If the best score is below 0.08 the system declines instead of guessing.
- Confidence is bucketed from the top score: high (>= 0.35), medium (>= 0.18), low (>= 0.08).
- Structured facts from the extraction (for example who provides or consumes a signal) are added next to the retrieved text.

- With an LLM, the prompt says: answer using ONLY the numbered excerpts, cite as [n], and say so if the excerpts do not contain the answer; temperature 0. Without an LLM, the best-matching sentences are returned with their [n] citations. If the LLM call fails, the system falls back to extractive mode.
- Every answer returns citations (document, version, page, section, score, excerpt) and warns when a source document is not approved.

4.2 Review and approval workflow

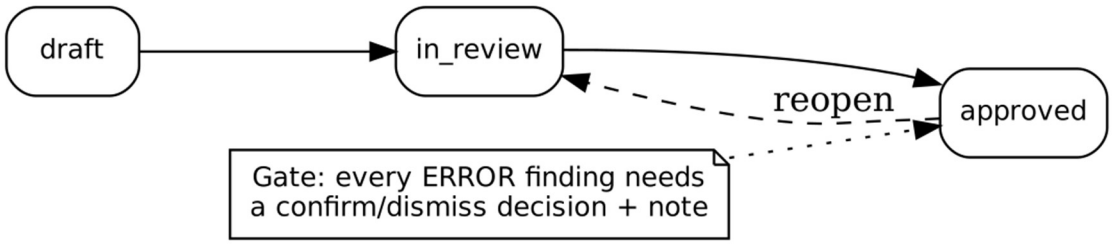


Figure 3. Document status workflow with approval gate

Engineers confirm or dismiss each finding with a note. The service refuses to move a document to approved while any error-level finding has no decision. All ingests, questions, reviews, status changes and exports are written to an audit log.

4.3 Consistency rules

Findings produced by checks.py include: undefined interface, component or port; port without interface; data-type mismatch and interface mismatch across a connection; signal type conflict; wrong connection direction; unconnected require/provide ports; dependency without connection; duplicate port; dependency cycle; producer/consumer rate mismatch; terminology inconsistency; component without runnable. Requirement-level checks add untraced requirements and timing budgets that are exceeded.

4.4 Setup and configuration

```
pip install -r requirements.txt
streamlit run app.py                          # UI; sidebar > Load sample documents
uvicorn hld_assistant.api:app --port 8000     # REST API (docs at /docs)
python -m hld_assistant analyze hld.pdf       # CLI / CI gate; exit 1 on errors
python -m hld_assistant compare old.pdf new.pdf
pytest                                        # 27 tests
```

Optional LLM: set HLD_LLM_URL (for example http://localhost:11434/v1), HLD_LLM_MODEL (for example llama3.1) and, if needed, HLD_LLM_KEY. State is stored in data/hld.sqlite.

5. Evaluation Steps and Results

5.1 Method

- Automated tests: 27 pytest tests covering ingestion, extraction, rules, comparison, Q&A, review gating and export (all passing on the delivered code).
- Seeded-defect check: v1 of the sample HLD contains known defects; we verify each is reported with the right rule and citation.

- Q&A probe: five questions run against v1 in extractive mode, including one out-of-scope question to test refusal.
- Revision comparison: v1 against v2 to verify structural and value-change detection.

5.2 Results - analysis of the sample documents

Document	Errors	Warnings	Info	Requirements	Test cases
brake_system_hld_v1.txt	5	1	3	1	10
brake_system_hld_v2.txt	0	0	4	1	14
brake_system.arxml	0	1	3	0	8

The five errors found in v1 are DATATYPE_MISMATCH (WheelSpeedProvider.WheelSpeedOut uint32 against BrakeController.WheelSpeedIn uint16), INTERFACE_MISMATCH (IfBrakeTorque connected to IfActuatorCmd), SIGNAL_TYPE_CONFLICT on WheelSpeed, UNDEFINED_COMPONENT (DiagManager) and UNDEFINED_INTERFACE (IfActuatorCmd). Version 2 resolves all errors. An estimated source-to-sink path WheelSpeedProvider -> BrakeController -> ActuatorDriver gives a rough sampling latency of 17 ms against a stated 20 ms requirement.

5.3 Results - Q&A probe (v1, extractive mode)

Question	Confidence	Outcome	Assessment
Who provides WheelSpeed?	high	Answered	Relevant P-Port and interface lines cited (p.1, sections 2.1 and 3.1). Correct but not phrased as one sentence.
What is the maximum latency requirement?	low	Answered	Returned the 20 ms end-to-end requirement from section 5. Correct; confidence label is conservative.
Which components depend on BrakeController?	medium	Answered	Cited connection lines and component text; the dependency statement itself is only partly surfaced.
What is the period of BrakeController?	medium	Weak	Retrieved connection lines rather than the runnable line (period 10 ms). Retrieval miss - see limitations.
What is the weather in Paris?	none	Declined	Correctly refused; no citations returned.

Across the five questions, 4 were answered and 1 was declined (answer rate 80%). Every answered question carried citations (groundedness 100% by the tool's metric, which checks that an answer has a cited source, not that the answer is fully correct). By manual reading, 3 of the 4 answers were fully satisfactory and 1 was a retrieval miss. This is a small hand-made probe, not a benchmark.

5.4 Results - revision comparison (v1 to v2)

The comparison reported: added component VehicleSpeedEstimator, added interface IfActuatorCmd, changed port ActuatorDriver.TorqueIn interface (IfActuatorCmd to IfBrakeTorque), changed WheelSpeedProvider.WheelSpeedOut type (uint32 to uint16), a new connection from VehicleSpeedEstimator to BrakeController, runnable changes (Run_BrakeCtrl_10ms replaced by Run_BrakeCtrl_5ms), and value changes in text (10 ms to 5 ms; latency requirement 20 ms to 15 ms).

5.5 Built-in metrics

The Metrics tab computes finding precision from reviewer decisions (confirmed / (confirmed + dismissed)), requirement traceability, answer rate and groundedness. Finding precision on the sample data is not yet available because no review decisions have been recorded; the reviewer should confirm or dismiss the findings during the demo to populate it.

5.6 Limitations observed

- The text extractor depends on the conventions in sample_data/; other layouts need pattern tuning. ARXML is the robust path.
- TF-IDF retrieval can miss short factual lines (the BrakeController period question); embedding-based retrieval would help.
- Latency estimates account for sampling delay only and are a screening aid, not timing analysis.
- Evaluation used the three sample documents; no real industrial HLD has been tested.
- No OCR, no diagram or table extraction from PDFs.

6. Responsible AI Measures

Area	Measure adopted
Grounding	Every answer, entity and finding cites document, version, page, section and evidence text. The LLM prompt is restricted to retrieved excerpts at temperature 0. If retrieval confidence is below threshold the system declines. Confidence level is shown with each answer.
Privacy	Documents and the SQLite database stay on the local machine. The only outbound call is the optional LLM endpoint, which can be a locally hosted model (Ollama, vLLM, llama.cpp). With no LLM configured, no document text leaves the machine.
Security	Parsing is local and deterministic; the LLM output is never executed. Limitation: there is no authentication or role-based access control yet, and the reviewer name is self-declared; projects are separated by name only. These should be added before multi-user deployment.
Human oversight	Findings are candidates, not verdicts. Reviewers confirm or dismiss each with a note; approval is blocked until every error has a decision; unapproved source documents trigger a warning in answers; the HTML report has a sign-off block.
Transparency & audit	Audit log records every ingest, question, review, status change and export. Rule-based findings are reproducible.
Fairness / misuse	The tool does not process personal data. It is a review aid and is not a substitute for ISO 26262 safety analysis or formal timing analysis; this is stated in the UI caption and the README.

7. Innovation Highlights

- **Citation-first design:** provenance (page, section, evidence) is stored for every extracted entity, so findings and answers can be verified in one click.
- **Hybrid RAG:** free-text retrieval is combined with structured facts from the extracted architecture model (providers, consumers, component summaries).
- **Identifier-aware tokenizer:** CamelCase and snake_case AUTOSAR names stay searchable as whole identifiers and as word parts.

- **Domain-specific checks:** AUTOSAR-aware rules such as provider/receiver direction, interface compatibility, signal type conflicts, rate mismatch and dependency cycles.
- **Value-aware revision diff:** detects numeric changes in text (20 ms to 15 ms) as well as structural changes.
- **Gated approval workflow:** human decisions on all errors are enforced in code, not left to convention.
- **Traced test generation:** test-case skeletons are linked to the components, signals and requirements they verify.
- **Works offline and as a CI gate:** extractive mode needs no LLM; the CLI returns a non-zero exit code on errors.

8. Conclusion and Future Work

The assistant turns a static HLD into a searchable, checkable and reviewable model, with every output traceable to its source and every design decision left to the engineer. Future work: embedding-based retrieval (FAISS or ChromaDB with BGE/E5), an LLM-based extractor feeding the same schema, OCR and table extraction, authentication with role-based access, a labelled Q&A benchmark on real HLDs, and proper timing analysis.

References and Third-Party Components

- Python libraries: streamlit, fastapi, uvicorn, scikit-learn, pandas, pypdf, python-docx, requests, reportlab, pytest, httpx (see requirements.txt).
- AUTOSAR Classic Platform concepts: software components, ports, interfaces, runnables, ARXML.
- Optional LLM runtimes: any OpenAI-compatible server (for example Ollama, vLLM, llama.cpp).

Appendix Submission Checklist

- Technical report exported to PDF from this document
- Signed academic integrity and AI-tool usage declarations (every student)
- GitHub repo shared: /report, /declarations, /code, /sample_data